



PRÁTICA 3 – Comunicação Serial - Escrevendo driver serial para STM32 - **Uso da função “printf”, e Interrupção Serial**

Prof. Imbiriba

1. Introdução

Comunicação serial é uma forma de transferência de dados entre dispositivos eletrônicos, que envolve a transmissão de bits de informação um de cada vez, em uma ordem específica. Em microcontroladores, a comunicação serial é utilizada para permitir a troca de informações com outros dispositivos, como sensores, atuadores, displays, módulos de comunicação sem fio, entre outros. Existem diferentes protocolos de comunicação serial que podem ser usados em microcontroladores, como USART/UART, SPI e I2C. Cada protocolo possui características e vantagens específicas, e a escolha do protocolo mais adequado depende das necessidades do projeto.

A UART (Universal Asynchronous Receiver/Transmitter) é um tipo de protocolo de comunicação serial assíncrono que utiliza apenas duas linhas de comunicação, uma para transmissão (TX) e outra para recepção (RX). Ele é usado para transferir dados em tempo real e é comumente utilizado em aplicações de comunicação serial entre um microcontrolador e um computador ou outro dispositivo.

USART significa "Universal Synchronous/Asynchronous Receiver/Transmitter" e é um módulo de comunicação serial encontrado em muitos microcontroladores. A USART combina as funcionalidades da UART (Universal Asynchronous Receiver/Transmitter), que é usada para comunicação serial assíncrona, e da SPI (Serial Peripheral Interface), que é usada para comunicação serial síncrona e possui protocolos de comunicação específicos para cada modo conforme mostrado na Figura1. Isso significa que a USART pode ser configurada para trabalhar tanto em modo síncrono quanto em modo assíncrono.

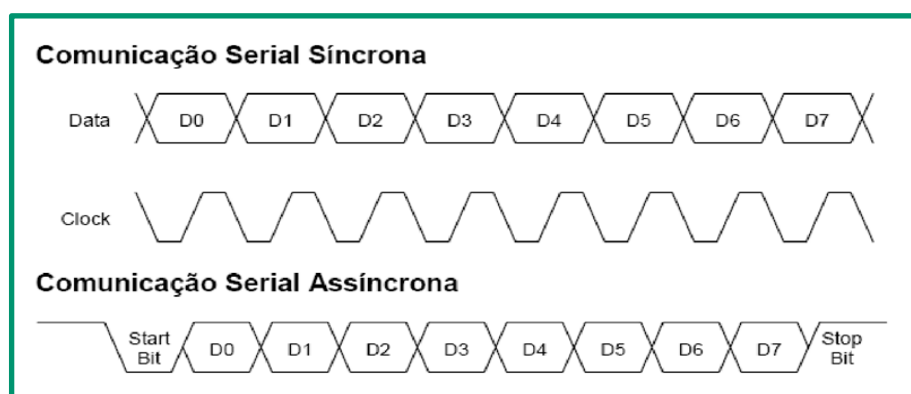


Figura 1 – Protocolos de comunicação serial

Em modo assíncrono, a USART é usada para a comunicação serial de dados em tempo real entre o microcontrolador e outros dispositivos externos, como sensores, atuadores, displays, módulos de comunicação sem fio, entre outros. Ele usa apenas duas linhas de comunicação, uma para transmitir dados (TX) e outra para receber dados (RX).



Em modo síncrono, a USART é usada para a transferência de grandes quantidades de dados em alta velocidade, e usa basicamente uma (Half-duplex) ou duas linhas (full-duplex) de comunicação: TX (Transmit), RX (Receive), além dos sinais de controle CLK (Clock) e algumas vezes o CS (Chip Select). A USART é capaz de enviar e receber dados de forma síncrona com outros dispositivos, como memórias, periféricos e outros microcontroladores.

Além disso, a USART é capaz de detectar e corrigir erros de transmissão, como paridade e erros de CRC, o que aumenta a confiabilidade da comunicação serial. A USART é uma opção muito versátil para comunicação serial em microcontroladores, oferecendo uma ampla variedade de opções de configuração para atender às necessidades específicas de cada projeto.

Para utilizar a comunicação serial em um microcontrolador, é necessário configurar os registradores adequados para cada protocolo, e escrever o código necessário para enviar e receber os dados de forma correta. Além disso, é importante estar atento às limitações de velocidade e distância de cada protocolo, para garantir uma comunicação confiável e estável entre os dispositivos envolvidos.

2. Comunicação USART na placa NÚCLEO

A interface **USART2** está disponível nos **pinos PA2 (TX) e PA3 (RX)** do microcontrolador STM32 e pode ser conectada ao MCU ST-LINK, ao conector ST morpho ou ao conector ARDUINO®. A escolha pode ser alterada configurando as pontes de solda relacionadas. **Por padrão, a comunicação USART2 entre o STM32 e o MCU ST-LINK é habilitada**, a fim de suportar a porta COM virtual para Mbed™ (SB13 e SB14 ligados, SB62 e SB63 desligados). Se a comunicação entre o STM32 PA2 (D1) ou PA3 (D0) e uma placa SHIELD ou extensão for necessária, SB62 e SB63 devem ser ligados, enquanto SB13 e SB14 devem ser desligados. Nesse caso, é possível conectar outra USART ao MCU ST-LINK usando fios aéreos entre o conector ST morpho e CN3. Por exemplo, no NUCLEO-F103RB, é possível usar a USART3 disponível em PC10 (TX) e PC11 (RX). Dois fios aéreos precisam ser conectados da seguinte maneira:

- PC10 (USART3_TX) disponível no pino 1 do CN7 para o pino RX do CN3
- PC11 (USART3_RX) disponível no pino 2 do CN7 para o pino TX do CN3

3. Implementação Prática

Como a USART2 é usada por padrão para comunicação entre o STM32 e o MCU ST-LINK, ela pode ser usada também para comunicar a aplicação com o PC, já que a interface ST-LINK da placa se comunica com o PC via interface USB. Assim usaremos essa interface para transmitir e receber dados seriais pelo PC. Para visualização dos dados recebidos e transmissão pela serial iremos usar um aplicativo terminal para Windows chamado HERCULES rodando no PC.

PARTE 1 – Desenvolvendo um driver de transmissão para o STM32:

- 1- Observe o diagrama de blocos do STM32 (DS pag. 15) e veja que a **UART2** está conectada ao barramento de periféricos APB1 do STM32 conforme pode ser visto na Figura2a. Portanto, será necessário habilitar o clock para a UART2 através do barramento de periféricos APB1.

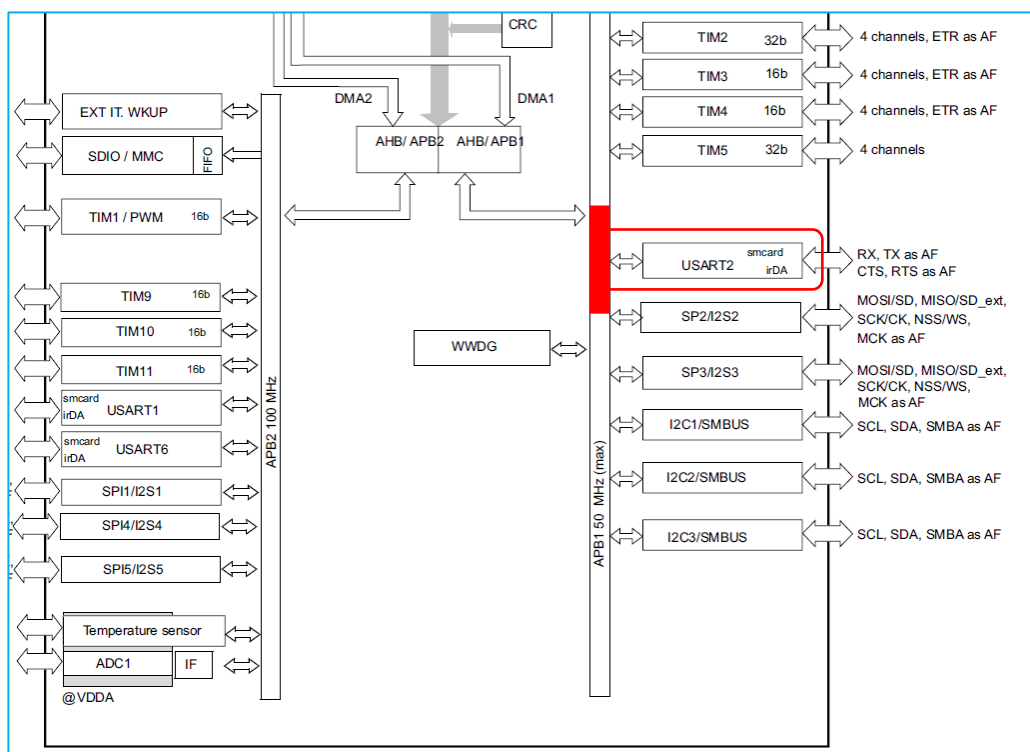


Figura2a – Diagrama de blocos do STM32 mostrando a interface USART2.

Para habilitação é necessário configurar o registrador de habilitação **RCC_APB1ENR** e setar o bit correspondente à habilitação da interface USART2. Lembre-se que o mesmo deve ser feito para a interface GPIOA através do registrador **RCC_AHB1ENR**, já que o pino correspondente à transmissão serial é o **PA2** (pino 2 do porta A).

6.3.11 RCC APB1 peripheral clock enable register (RCC_APB1ENR)

Address offset: 0x40

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			PWR EN	Reserved				I2C3 EN	I2C2 EN	I2C1 EN	Reserved			USART2 EN	Reserved
			rw					rw	rw	rw				rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 EN	SPI2 EN	Reserved		WWDG EN	Reserved							TIM5 EN	TIM4 EN	TIM3 EN	TIM2 EN
rw	rw			rw								rw	rw	rw	rw

Figura2b – Registrador para habilitação do clock para uso da USART2.

- Como o pino PA2 é usado como ALTERNATE FUNCTION (pino de transmissão da UART2), ele deverá ser configurado como tal no registrador GPIO port mode register **GPIOx_MODER** – RM pag.157, conforme mostrado na Figura3.



8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..E and H)

Address offset: 0x00

Reset values:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]				MODER14[1:0]				MODER13[1:0]				MODER12[1:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]				MODER6[1:0]				MODER5[1:0]				MODER4[1:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
MODER3[1:0]				MODER2[1:0]				MODER1[1:0]				MODER0[1:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Figura3 – Configuração do modo de operação do GPIOA.

- 3- Para configurar a função alternativa relativa ao pino PA2 como sendo o pino de transmissão serial para a UART2, é necessário configurar o registro **GPIO alternate function low** – RM pag 161, conforme a Figura4.

8.4.9 GPIO alternate function low register (GPIOx_AFRL) (x = A..E and H)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:0 **AFRLy**: Alternate function selection for port x bit y (y = 0..7)

These bits are written by software to configure alternate function I/Os

AFRLy selection:

0000: AF0

0001: AF1

0010: AF2

0011: AF3

0100: AF4

0101: AF5

0110: AF6

0111: AF7

1000: AF8

1001: AF9

1010: AF10

1011: AF11

1100: AF12

1101: AF13

1110: AF14

1111: AF15

Figura4 – Configuração da função alternativa de PA2 para AF07 (0111).

A configuração AF7 determina que o pino PA2 terá como função alternativa a transmissão serial da UART2 - TX. Essa informação pode ser obtida do mapa de funções alternativas do DATASHEET do STM32 – DS pag. 47, conforme mostrado na Figura5.



Table 9. Alternate function mapping

Port	AF00	AF01	AF02	AF03	AF04	AF05	AF06	AF07	AF08	AF09	AF10	AF11	AF12	AF13	AF14	AF15
	SYS_AF	TIM1/TIM2	TIM3/TIM4/TIM5	TIM9/TIM10/TIM11	I2C1/I2C2/I2C3	SPI1/I2S1S PI2/I2S2/SPI3/I2S3	SPI2/I2S2/SPI3/I2S3/SPI4/I2S4/SPI5/I2S5	SPI3/I2S3/USART1/USART2	USART6	I2C2/I2C3	OTG1_FS		SDIO			
PA0	-	TIM2_CH1/TIM2_ETR	TIM5_CH1	-	-	-	-	USART2_CTS	-	-	-	-	-	-	-	EVENT OUT
PA1	-	TIM2_CH2	TIM5_CH2	-	-	SPI4_MOSI/I2S4_SD	-	USART2_RTS	-	-	-	-	-	-	-	EVENT OUT
PA2	-	TIM2_CH3	TIM5_CH3	TIM9_CH1	-	I2S2_CKIN	-	USART2_TX	-	-	-	-	-	-	-	EVENT OUT
PA3	-	TIM2_CH4	TIM5_CH4	TIM9_CH2	-	I2S2_MCK	-	USART2_RX	-	-	-	-	-	-	-	EVENT OUT
PA4	-	-	-	-	-	SPI1_NSS/I2S1_WS	SPI3_NSS/I2S3_WS	USART2_CK	-	-	-	-	-	-	-	EVENT OUT
PA5	-	TIM2_CH1/TIM2_ETR	-	-	-	SPI1_SCK/I2S1_CK	-	-	-	-	-	-	-	-	-	EVENT OUT
PA6	-	TIM1_BKIN	TIM3_CH1	-	-	SPI1_MISO	I2S2_MCK	-	-	-	-	-	SDIO_CMD	-	-	EVENT OUT
PA7	-	TIM1_CH1N	TIM3_CH2	-	-	SPI1_MOSI/I2S1_SD	-	-	-	-	-	-	-	-	-	EVENT OUT
PA8	MCO_1	TIM1_CH1	-	-	I2C3_SCL	-	-	USART1_CK	-	-	USB_FS_SOF	-	SDIO_D1	-	-	EVENT OUT

Figura5 – Obtenção da função alternativa de PA2 para AF07.

- Habilitação da interface USART e da transmissão serial – “Control register 1 (USART_CR1)” – RM pag. 550.

O registrador **CR1** possui um conjunto de bits para controle do canal serial, conforme mostrado na Figura6.

19.6.4 Control register 1 (USART_CR1)
Address offset: 0x0C
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

1 1 1 1

550/844 RM0383 Rev 3

Figura6 – Registrador de controle da interface serial - USART

Para a habilitação da UART e a habilitação da transmissão serial é necessário a colocação de 1 nos bits UE e TE respectivamente do registrador CR1. O bit RE é usado para habilitar a recepção de dados pela UART e será usado na PARTE4. O bit RXNEIE é usado para habilitar a interrupção de recepção serial e será usado na PARTE5.

- Registrador de STATUS da interface serial USART.

O registrador **SR** – “Status register (USART_SR)” – RM pag. 547, possui um conjunto de bits para informar o status da interface serial, conforme pode ser visto na Figura7. Nessa prática, nós iremos ler o STATUS do bit **TXE** que sinaliza que o buffer de transmissão está vazio e um byte pode ser enviado para transmissão. O bit **RXNE** sinaliza a recepção de um dado pela serial.



19.6.1 Status register (USART_SR)

Address offset: 0x00

Reset value: 0x00C0 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE
Reserved						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

Figura7 – Registrador status da interface serial - USART

6. Registrador de dados da interface serial – “Data register (USART_DR)”.

O registrador **DR** “Data register (USART_DR)” contém o dado a ser transmitido ou recebido pela interface serial. Portanto, para o envio de um dado pela USART basta escrever o dado a ser enviado nesse registrador. Na recepção o dado é armazenado nesse registrador.

7. Configuração da taxa de transferência de bits (BAUDRATE) para UART2: BAUDRATE – Taxa de transferência de bits o projeto: 9600 bps;

A definição do BAUDRATE é realizada mediante valor escrito no **registrador “Baud rate register (USART_BRR)”** – RM pag.550, mostrado na Figura8.

19.6.3 Baud rate register (USART_BRR)

Note: The baud counters stop counting if the TE or RE bits are disabled respectively.

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_Mantissa[11:0]												DIV_Fraction[3:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:16: Reserved, must be kept at reset value

Bits 15:4 **DIV_Mantissa[11:0]**: mantissa of USARTDIV
 These 12 bits define the mantissa of the USART Divider (USARTDIV)

Bits 3:0 **DIV_Fraction[3:0]**: fraction of USARTDIV
 These 4 bits define the fraction of the USART Divider (USARTDIV). When OVER8=1, the DIV_Fraction3 bit is not considered and must be kept cleared.

Figura8 - Registrador USART_BRR que define o BAUDRATE serial.

O cálculo do valor em função do BAUDRATE pode ser expressado da seguinte forma:

$$\text{Valor} = (\text{frequência do clock} + \text{BAUDRATE}/2) / \text{BAUDRATE}$$



8. Abra o STM32CUBEIDE. Faça uma cópia do projeto anterior (CTRL+C) e copie (CTRL+V) na pasta de trabalho (workspace). Renomeie o projeto copiado para **2_serial_usart**;
9. Edit o **main.c** de acordo com o código a seguir:

```
1 #include <stdint.h>
2 #include "stm32f4xx.h"
3
4 #define GPIOAEN      (1U<<0)
5 #define UART2EN      (1U<<17)
6
7 #define CR1_TE        (1U<<3)
8 #define CR1_UE        (1U<<13)
9 #define SR_TXE        (1U<<7)
10
11 #define SYS_FREQ      16000000 /* Constante do clock da CPU */
12 #define APB1_CLK      SYS_FREQ
13
14 #define UART_BAUDRATE 9600
15
16 /* Declarações das funções protótipos */
17 static void uart_set_baudrate (USART_TypeDef *USARTx, uint32_t PeriphClk, uint32_t BaudRate);
18 static uint16_t compute_uart_bd (uint32_t PeriphClk, uint32_t BaudRate);
19 void uart2_tx_init(void);
20 void uart2_write(int ch);
21
22 int main(void)
23 {
24     uart2_tx_init();
25
26     while(1)
27     {
28         uart2_write('x');
29     }
30 }
31
32 void uart2_tx_init(void)
33 {
34     /****** Configura o pino GPIO para UART *****/
35     /* Habilita o clock para acessar GPIOA */
36     RCC->AHB1ENR |= GPIOAEN;
37
38     /* Configura o modo de PA2 para função alternativa */
39     GPIOA->MODER &=~(1U<<4);
40     GPIOA->MODER |= (1U<<5);
41
42     /* Configura PA2 para a função alternativa UART_TX (AF07)*/
43     GPIOA->AFR[0] |= (1U<<8);
44     GPIOA->AFR[0] |= (1U<<9);
45     GPIOA->AFR[0] |= (1U<<10);
46     GPIOA->AFR[0] &= ~(1U<<11);
47
48     /******Configura o canal serial uart2 *****/
49     /* Habilita o clock para acessar a uart2 */
50     RCC->APB1ENR |= UART2EN;
51
52     /* Configura o baudrate */
53     uart_set_baudrate(USART2,APB1_CLK,UART_BAUDRATE);
54
55     /* Habilita a transmissão serial*/
56     USART2->CR1 = CR1_TE;
57
58     /* Habilita a uart2 */
59     USART2->CR1 |= CR1_UE;
60 }
```




```
61
62 void uart2_write(int ch)
63 {
64     /* Testa se o buffer de transmissão está vazio */
65     while(!(USART2->SR & SR_TXE)){
66
67     /* Escreve dado no registrador de saída da uart2 (data register) */
68     USART2->DR = (ch & 0xFF);
69 }
70
71 static void uart_set_baudrate (USART_TypeDef *USARTx, uint32_t PeriphClk, uint32_t BaudRate)
72 {
73     USARTx->BRR = compute_uart_bd(PeriphClk,BaudRate);
74 }
75
76 static uint16_t compute_uart_bd (uint32_t PeriphClk, uint32_t BaudRate)
77 {
78     return ((PeriphClk + (BaudRate/2U))/BaudRate);
79 }
80
```

main.c

10. Compile e carregue o código na placa Núcleo.
11. Rode ou faça o DEBUG do código e observe o resultado no terminal serial HERCULES.

PARTE 2 – Usando o driver serial com a função printf da biblioteca stdio.h

1. Abra o STM32CUBEIDE. Faça uma cópia do projeto anterior (CTRL+C) e copie (CTRL+V) na pasta de trabalho (workspace). Renomeie o projeto copiado para **3_printf_usart**;
2. Edit o **main.c** introduzindo o código a seguir antes da função main:

```
23 /* Redefinição da função de escrever um dado/caractere pela serial */
24 int __io_putchar(int ch)
25 {
26     uart2_write(ch);
27     return ch;
28 }
29
30 int main(void)
31 {
32     uar2_tx_init();
33     while(1)
34     {
35         printf("Olá, Unifor - STM32F4.....\n\r");
36     }
37 }
```

main.c

A função **putchar()** é a função nativa da biblioteca stdio.h que envia um byte pela serial quando usamos a função **printf**. A função **printf** envia um conjunto de caracteres (string) pelo dispositivo de saída padrão, nesse caso, a porta serial. A introdução do código acima irá substituir a chamada da função nativa **putchar()** pela função que criamos no código acima, sempre que formos usar o **printf**. Dessa forma, usaremos o DRIVER serial que escrevemos juntamente com a função **printf** para enviar uma string pela serial.



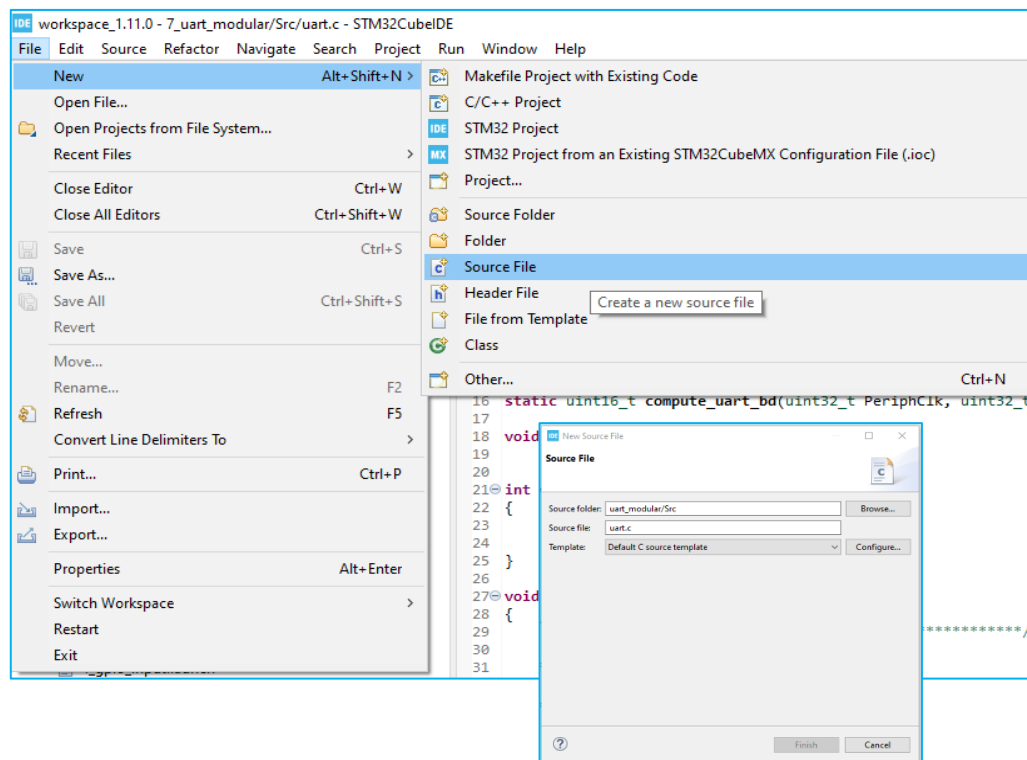
3. Faça a inclusão da biblioteca **stdio.h** no início do código.

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include "stm32f4xx.h"
4
5 #define GPIOAEN      (1U<<0)
6 #define UART2EN      (1U<<17)
7
```

4. Compile e carregue o código na placa Núcleo.
5. Rode ou faça o DEBUG do código e observe o resultado no terminal serial HERCULES.

PARTE 3 – UART modular. Inclusão dos arquivos header do driver serial.

1. Abra o STM32CUBEIDE. Faça uma cópia do projeto anterior (CTRL+C) e copie (CTRL+V) na pasta de trabalho (workspace). Renomeie o projeto copiado para **4_uart_modular**;
2. Vá em **File->New->Source File** e crie um novo arquivo fonte chamado **uart.c** na pasta **Src** do diretório de trabalho, conforme mostrado a seguir:



3. Edit o arquivo **uart.c** como mostrado descrito abaixo:

Copie todo o conteúdo editado do arquivo **main.c** da **PARTE1** juntamente com a função **[int __io_putchar(int ch)]** da PARTE2, com exceção da função **main**: **[int main (void)]**.

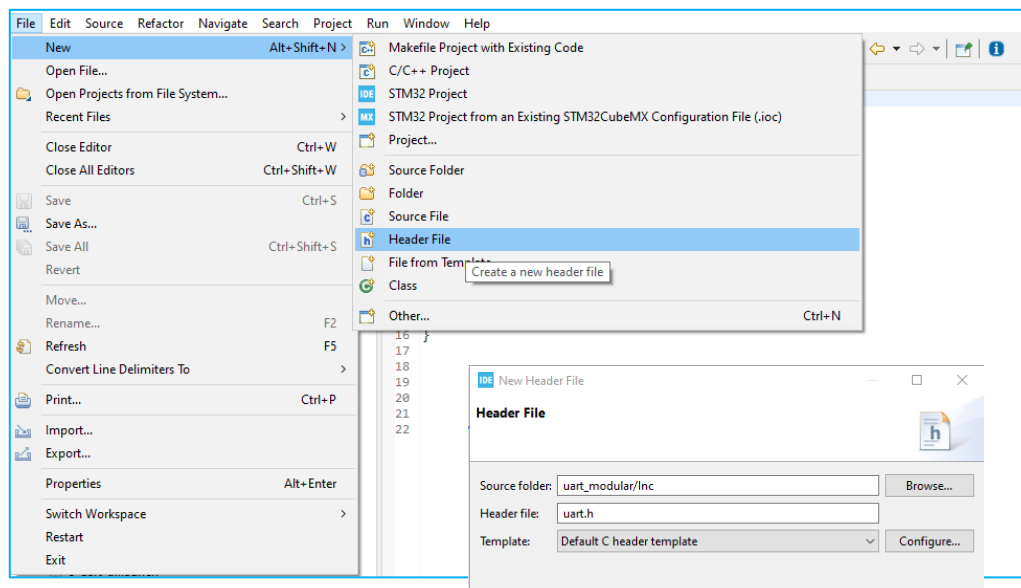


O arquivo **uart.c** deve conter o código de todas as funções já descritas com a exceção da função **main**, e o include do arquivo **uart.h**, tal como mostrado abaixo:

```
1 #include "uart.h"
2
3 #define GPIOAEN      (1U<<0)
4 #define UART2EN      (1U<<17)
5
6 #define CR1_TE        (1U<<3)
7 #define CR1_UE        (1U<<13)
8 #define SR_TXE        (1U<<7)
9
10 #define SYS_FREQ      16000000
11 #define APB1_CLK      SYS_FREQ
12
13 #define UART_BAUDRATE 115200
14
15 /* Declarações das funções protótipos */
16 static void uart_set_baudrate(USART_TypeDef *USARTx, uint32_t PeriphClk,  uint32_t BaudRate);
17 static uint16_t compute_uart_bd(uint32_t PeriphClk, uint32_t BaudRate);
18 void uar2_tx_init(void);
19 void uart2_write(int ch);
20
21 /* Redefinição da função de escrever um dado/caractere pela serial */
22 int __io_putchar(int ch)
23 {
24     uart2_write(ch);
25     return ch;
26 }
27
28 void uart2_tx_init(void)
29 {
30     /******Configure uart gpio pin*****
31     /*Enable clock access to gpioa */
32     RCC->AHB1ENR |= GPIOAEN;
33
34     /*Set PA2 mode to alternate function mode*/
35     GPIOA->MODER &= ~(1U<<4);
36     GPIOA->MODER |= (1U<<5);
37
38     /*Set PA2 alternate function type to UART_TX (AF07)*/
39     GPIOA->AFR[0] |= (1U<<8);
40     GPIOA->AFR[0] |= (1U<<9);
41     GPIOA->AFR[0] |= (1U<<10);
42     GPIOA->AFR[0] &= ~(1U<<11);
43
44     /******Configure uart module *****
45     /*Enable clock access to uart2 */
46     RCC->APB1ENR |= UART2EN;
47
48     /*Configure baudrate*/
49     uart_set_baudrate(USART2,APB1_CLK,UART_BAUDRATE);
50
51     /*Configure the transfer direction*/
52     USART2->CR1 = CR1_TE;
53
54     /*Enable uart module*/
55     USART2->CR1 |= CR1_UE;
56 }
57
58 void uart2_write(int ch)
59 {
60     /*Make sure the transmit data register is empty*/
61     while(!(USART2->SR & SR_TXE)){
62
63     /*Write to transmit data register*/
64     USART2->DR = (ch & 0xFF);
65     }
66 }
67 static void uart_set_baudrate(USART_TypeDef *USARTx, uint32_t PeriphClk,  uint32_t BaudRate)
68 {
69     USARTx->BRR = compute_uart_bd(PeriphClk,BaudRate);
70 }
71
72 static uint16_t compute_uart_bd(uint32_t PeriphClk, uint32_t BaudRate)
73 {
74     return ((PeriphClk + (BaudRate/2U))/BaudRate);
75 }
76 }
```

uart.c

4. Vá em **File->New->Header File** e crie um novo arquivo cabeçalho chamado **uart.h** na pasta **Inc** do diretório de trabalho, conforme mostrado a seguir:



5. Edit o arquivo **uart.h** como descrito abaixo:

```
1 #ifndef UART_H_
2 #define UART_H_
3 #include <stdint.h>
4
5 #include "stm32f4xx.h"
6
7 #endif
8
```

uart.h

6. Edit o **main.c** deletando todo o seu conteúdo deixando apenas o código a seguir:

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include "stm32f4xx.h"
4 #include "uart.h"
5
6 void uart2_tx_init(void);
7
8 int main(void)
9 {
10     uart2_tx_init();
11
12     while(1)
13     {
14         printf("Hello from STM32F4.....\n\r");
15     }
16 }
17
```

main.c

Você deve fazer os includes dos arquivos cabeçalhos, inclusive o **uart.h** editado anteriormente.

7. Compile e carregue o código na placa Núcleo;
8. Rode ou faça o DEBUG do código e observe o resultado no terminal serial HERCULES;



PARTE 4 – Recepção Serial

1. Modifique o arquivo **uart.c** conforme descrito abaixo:

Renomeia a função **uart2_tx_init** para **uart2_rtx_init**, habilite a recepção serial, configure PA3 para a função alternativa UART_RX (AF07), e adicione a função de leitura da serial [**uart2_read**], conforme mostrado abaixo:

```
32 void uart2_rtx_init(void)
33 {
34     /****** Configura o pino GPIO para UART *****/
35     /* Habilita o clock para acessar GPIOA */
36     RCC->AHB1ENR |= GPIOAEN;
37
38     /* Configura o modo de PA2 para função alternativa */
39     GPIOA->MODER &= ~(1U<<4);
40     GPIOA->MODER |= (1U<<5);
41
42     /* Configura PA2 para a função alternativa UART_TX (AF07)*/
43     GPIOA->AFR[0] |= (1U<<8);
44     GPIOA->AFR[0] |= (1U<<9);
45     GPIOA->AFR[0] |= (1U<<10);
46     GPIOA->AFR[0] &= ~(1U<<11);
47
48     /* Configura PA3 para a função alternativa UART_RX (AF07)*/
49     GPIOA->MODER &= ~(1U<<6);
50     GPIOA->MODER |= (1U<<7);
51
52     /* Configura PA3 para a função alternativa UART_RX (AF07)*/
53     GPIOA->AFR[0] |= (1U<<12);
54     GPIOA->AFR[0] |= (1U<<13);
55     GPIOA->AFR[0] |= (1U<<14);
56     GPIOA->AFR[0] &= ~(1U<<15);
57
58     /******Configura o canal serial uart2 *****/
59     /* Habilita o clock para acessar a uart2 */
60     RCC->APB1ENR |= UART2EN;
61
62     /* Configura o baudrate */
63     uart_set_baudrate(USART2,APB1_CLK,UART_BAUDRATE);
64
65     /* Habilita a transmissão serial*/
66     USART2->CR1 = (CR1_TE | CR1_RE);
67
68     /* Habilita a uart2 */
69     USART2->CR1 |= CR1_UE;
70 }
71
72 char uart2_read(void)
73 {
74     /* Verifica se o buffer serial de recepção não está vazio*/
75     while(!(USART2->SR & SR_RXNE)){
76
77     /* Ler o dado */
78     return USART2->DR;
79 }
```

uart.c – função **uart2_rtx_init** modificada e inclusão da função **uart2_read**

Mantenha o restante do código abaixo da linha 32 em **uart.c**.

2. Edit o **main.c** conforme o código a seguir:



```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include "stm32f4xx.h"
4 #include "uart.h"
5
6 #define GPIOAEN      (1U<<0)
7 #define GPIOA_5      (1U<<5)
8
9 #define LED_PIN      GPIOA_5
10
11 char key;           /* Variável que vai receber o caractere */
12
13 int main(void)
14 {
15     /* Habilita o clock para acessar GPIOA */
16     RCC->AHB1ENR |= GPIOAEN;
17
18     /* Configura o pino PA5 como entrada - RX Serial */
19     GPIOA->MODER |= (1U<<10);
20     GPIOA->MODER &= ~(1U<<11);
21
22     /* Configura o canal serial - UART2 */
23     uart2_rxtx_init();
24
25     while(1)
26     {
27         key = uart2_read(); /* Variável que vai receber o caractere */
28         if(key == '1')      /* Se receber o caractere '1' toggle o led */
29         {
30             GPIOA->ODR ^= LED_PIN;
31         }
32         else{
33             GPIOA->ODR &= ~LED_PIN;
34         }
35     }
36 }
```

main.c

3. Compile e carregue o código na placa Núcleo;
4. Rode ou faça o DEBUG do código e digite o número 1 no teclado ou qualquer outra tecla no terminal serial HERCULES e observe o resultado.

PARTE 5 – Recepção Serial via Interrupção

1. Nessa etapa é necessário habilitar a interrupção de recepção do canal serial setando o bit **RXNEIE** em CR1 e a interrupção correspondente ao módulo serial no controlador NVIC. Modifique o arquivo **uart.c** adicionando as instruções destacadas em vermelho, conforme mostrado abaixo:

```
56 GPIOA->AFR[
57
58 #define CR1_UE      (1U<<13)
59 #define SR_TXE      (1U<<7)
60 #define CR1_RXNEIE  (1U<<5)
61 #define CR1_RE      (1U<<2)
62
63 /* Configura o canal serial uart2 */
64 /* Habilita o clock para acessar a uart2 */
65 RCC->APB1ENR |= UART2EN;
66
67 /* Configura o baudrate */
68 uart_set_baudrate(USART2,APB1_CLK,UART_BAUDRATE);
69
70 /* Habilita a transmissão e recepção serial */
71 USART2->CR1 = (CR1_TE | CR1_RE);
72
73 /* Habilita interrupção da recepção serial - RXNE */
74 USART2->CR1 |= CR1_RXNEIE;
75
76 /* Habilita interrupção serial no controlador NVIC */
77 NVIC_EnableIRQ(USART2_IRQn); /* ver RM pag. 203 */
78
79 /* Habilita a uart2 */
80 USART2->CR1 |= CR1_UE;
81 }
```

uart.c



2. Edit o **main.c** conforme o código a seguir:

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include "stm32f4xx.h"
4 #include "uart.h"
5
6 #define GPIOAEN          (1U<<0)
7 #define GPIOA_5          (1U<<5)
8
9 #define LED_PIN          GPIOA_5
10
11 char key;
12
13 static void uart_callback(void);
14 void uart2_rtx_init(void);
15
16 int main(void)
17 {
18     /* Habilita o clock para acessar o GPIOA */
19     RCC->AHB1ENR |= GPIOAEN;
20
21     /* Configura PA5 como pino de saída - LED */
22     GPIOA->MODER |= (1U<<10);
23     GPIOA->MODER &= ~(1U<<11);
24
25     /* Inicializa o canal serial */
26     uart2_rtx_init();
27
28     while(1)
29     {
30         /* nada */
31     }
32 }
33
34 /* Rotina de serviço da interrupção de recepção serial - callback */
35 static void uart_callback(void)
36 {
37     key = USART2->DR;
38     GPIOA->ODR ^= LED_PIN;
39     printf("tecla pressionada : %c \n\r",key);
40 }
41
42 /* Manipulador de interrupção do canal serial */
43 void USART2_IRQHandler(void)
44 {
45     /* Verifica se RXNE is set */
46     if(USART2->SR & SR_RXNE)
47     {
48         uart_callback();
49     }
50 }
51
52
```

main.c

Observe que agora nada é feito dentro do **while(1)**. Quando não existe nenhum dado recebido pela serial o microcontrolador não faz nada. Quando um dado é recebido pela serial uma interrupção acontece e o processador irá executar a função **USART2_IRQHandler**. Essa função por sua vez irá chamar a função call-back da serial - **uart_callback**, ou seja, a função que tratará o byte recebido pela serial.

3. Compile e carregue o código na placa Núcleo;
4. Rode ou faça o DEBUG do código, envie qualquer caractere pelo terminal serial HERCULES, e observe o resultado.